



Lab Assignment – 10

Department: CSE

Subject: DIS Lab
Code: 22CPP217

Objective: Design, normalize, and implement a complete, production-ready relational database system for a high-traffic E-commerce platform. This assignment requires the application of BCNF normalization, complex DDL/DML with referential actions, automated business logic using triggers and procedures, advanced analytical querying (CTEs, Lateral Joins), and strict transaction concurrency control.

Scenario: You are the lead database architect for *RigBuilder*, a specialized e-commerce platform selling high-end computer hardware (e.g., 4K gaming monitors, external solid-state drives, GPUs, and custom cooling systems). The system must handle thousands of concurrent users browsing the catalog, adding items to carts, processing payments, and managing volatile inventory levels.

Pre-Lab Setup: Run this block to initialize a Banking environment to simulate financial transactions.

SQL

```
-- 0. Disable Safe Update Mode & Reset Environment
SET SQL_SAFE_UPDATES = 0;

-- Clean up any existing tables from previous runs
DROP TABLE IF EXISTS Order_Items, Orders, Products, Customers;
DROP TABLE IF EXISTS Legacy_Sales_Data;

-- 1. Create the Denormalized Legacy Schema
CREATE TABLE Legacy_Sales_Data (
    Order_ID INT,
    Customer_Name VARCHAR(100),
    Customer_Email VARCHAR(100),
    Shipping_Address VARCHAR(255),
    Product_ID INT,
    Product_Name VARCHAR(100),
```

```

    Category VARCHAR(50),
    Stock_Quantity INT,
    Unit_Price DECIMAL(10, 2),
    Order_Date DATE,
    Order_Status VARCHAR(50),
    Qty_Purchased INT,
    PRIMARY KEY (Order_ID, Product_ID)
);

-- 2. Insert Denormalized Sample Data (Contains deliberate redundancies)
INSERT INTO Legacy_Sales_Data VALUES
(1001, 'Aarav Sharma', 'aarav.s@email.com', '12 JLN Marg, Jaipur', 501,
'Odysey G7 32" 4K Monitor', 'Monitors', 15, 65000.00, '2026-03-15',
'Completed', 1),
(1001, 'Aarav Sharma', 'aarav.s@email.com', '12 JLN Marg, Jaipur', 502,
'Samsung 990 PRO 2TB NVMe', 'Storage', 40, 18000.00, '2026-03-15', 'Completed',
2),
(1002, 'Priya Verma', 'priya.v@email.com', '45 Malviya Nagar, Jaipur', 503,
'NVIDIA RTX 4080 Super', 'GPUs', 5, 120000.00, '2026-03-16', 'Processing', 1),
(1003, 'Rohan Singh', 'rohan.s@email.com', '88 Ring Road, Delhi', 501, 'Odysey
G7 32" 4K Monitor', 'Monitors', 15, 65000.00, '2026-03-17', 'Shipped', 1),
(1004, 'Aarav Sharma', 'aarav.s@email.com', '12 JLN Marg, Jaipur', 504,
'Corsair iCUE H150i Cooler', 'Cooling', 25, 14500.00, '2026-03-18',
'Completed', 1),
(1005, 'Meera Joshi', 'meera.j@email.com', '10 Civil Lines, Pune', 502,
'Samsung 990 PRO 2TB NVMe', 'Storage', 40, 18000.00, '2026-03-18', 'Cancelled',
1);

```

Q 1: Architecture & Lossless Decomposition (Normalization)

You are given the following legacy, highly denormalized data structure currently used by the data migration team:

Legacy_Sales_Data (Order_ID, Customer_Name, Customer_Email, Shipping_Address, Product_ID, Product_Name, Category, Stock_Quantity, Unit_Price, Order_Date, Order_Status, Qty_Purchased)

1. **Decompose to BCNF:** Programmatically transform this single table into a robust, normalized schema (at least 3NF/BCNF). You should logically separate Customers, Products, Orders, and Order_Items.

2. **Schema Definition (DDL):** Write the **CREATE TABLE** statements for your normalized schema.
 - Enforce strict integrity: Ensure **Customer_Email** is **UNIQUE**.
 - Apply a **CHECK** constraint to ensure product **Stock_Quantity** can never fall below zero.
 - Implement referential integrity: If an **Order** is cancelled and deleted from the system, all associated **Order_Items** must be removed automatically using **ON DELETE CASCADE**.

Q 2: Database Automation (Triggers & Procedures)

The system must maintain strict inventory and operational guidelines without relying solely on the application layer.

1. **The Inventory Failsafe (Trigger):** Overselling hardware during a flash sale is a critical business risk. Create a **BEFORE INSERT** trigger on your **Order_Items** table. If the **Qty_Purchased** exceeds the available **Stock_Quantity** in the **Products** table, the trigger must block the transaction and throw a custom error using **SIGNAL SQLSTATE**.
2. **Automated Refund Processing (Stored Procedure):** Create a stored procedure named **Process_Refund(IN p_Order_ID)**. This procedure must automatically:
 - Update the **Order_Status** to 'Refunded'.
 - Join with the **Order_Items** table to restore the exact purchased quantities back into the **Products** inventory.
 - Log this action into a separate **Financial_Audit_Log** table.

Q 3: Advanced Analytical Querying

The marketing and procurement teams need complex reports to analyze consumer behaviour and hardware sales trends.

1. **Customer Loyalty Categorization:** Write a query using a **CASE** statement to categorize customers into loyalty tiers based on their total lifetime spend :
 - Platinum: Spent more than \$2,000.
 - Gold: Spent more than \$1,000 (but \$2,000 or less).
 - Silver: Everyone else (spent \$1,000 or less).
2. **Revenue Analysis (CTEs):** Use a Common Table Expression (**WITH**) to calculate the total revenue generated per product **Category** (e.g., 'Monitors', 'Storage') . Query the CTE to display only the categories that have generated more than \$50,000 in total sales.
3. **Deep Purchasing Habits (LATERAL Joins):** For every registered customer, retrieve exactly their *two most expensive* individual orders. You must use a **JOIN LATERAL** (or Window Functions) to accomplish this row-dependent sorting and limiting.

Q 4: Security & Transactional Concurrency

You must secure the catalog and ensure that concurrent checkout processes do not corrupt inventory levels.

1. **Role-Based Access Control:** Create a View named `Public_Hardware_Catalog` that exposes only the `Product_Name`, `Category`, and `Unit_Price` (strictly hiding sensitive supplier costs and exact warehouse stock numbers) . `GRANT` read-only (`SELECT`) access on this view to a newly created user role named `Frontend_App`.
2. **Preventing Lost Updates (Race Conditions):** Two customers are attempting to buy the very last high-refresh-rate gaming monitor at the exact same millisecond.
 - Write a SQL script utilizing `START TRANSACTION` and `SELECT ... FOR UPDATE` to lock the specific monitor's row while checking its stock.
 - Deduct the inventory safely, then `COMMIT` to release the lock for other transactions.